

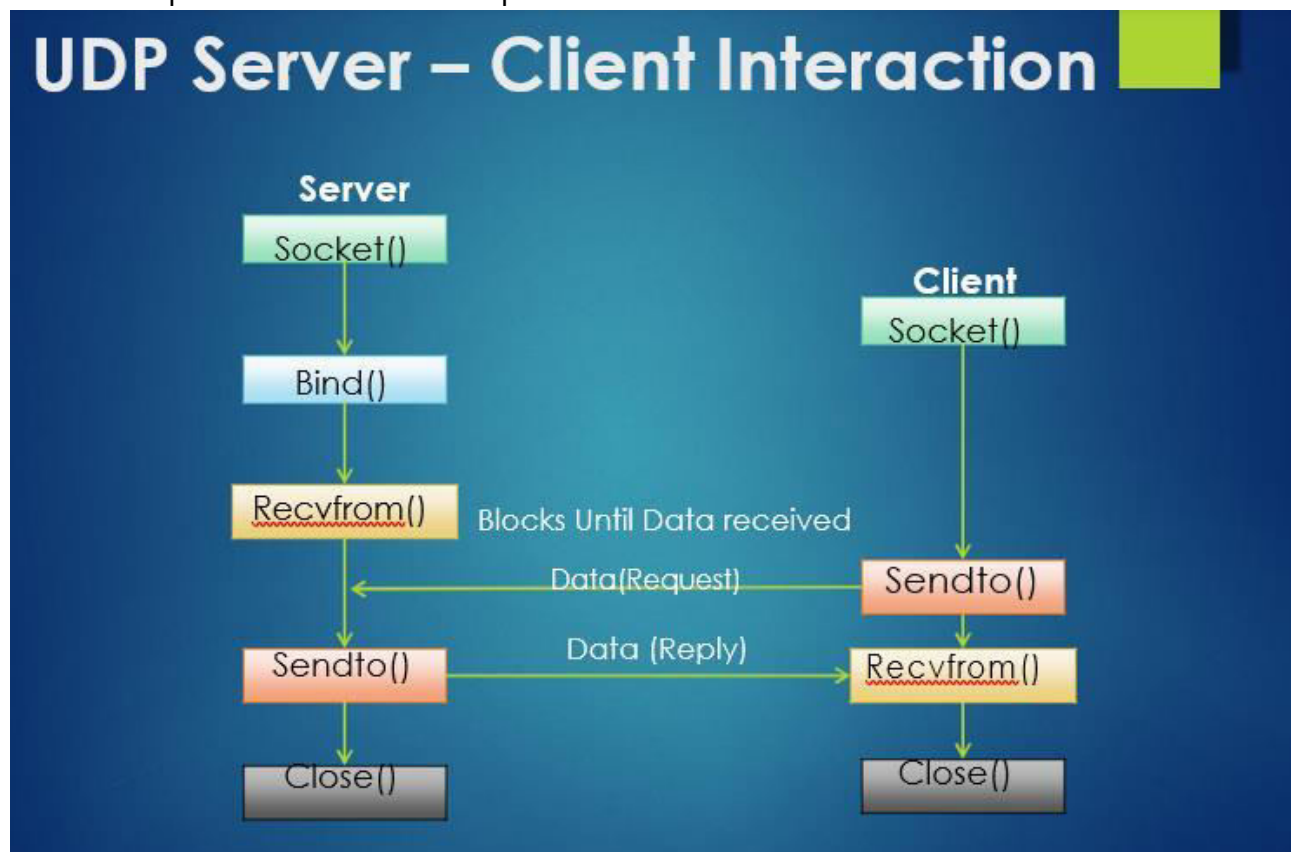
Socket Programming: Concurrent Servers

Note: We understand that many of you have not studied OS yet and because of that we've include some basic explanations of the concepts that you may need to implement concurrency for UDP.

1. Introduction to Concurrent UDP Servers

By now you should be familiar with basic TCP and UDP servers and clients. In previous labs, we used the **iterative server model** where:

- Server serves one client at a time
- Other clients must wait in queue
- Simple but inefficient for multiple clients



The Problem with Iterative Servers

Client 1 → Server (serving...) → Client 2 (waiting...) → Client 3 (waiting...)

Issues:

- Long waiting times for clients
- Server idle while waiting for one client
- Poor scalability for real-world applications
- Bottleneck in high-traffic scenarios

The Solution: Concurrent Servers

Concurrent servers can handle multiple clients simultaneously by creating separate execution contexts (processes or threads) for each client.

2. Concurrency Methods Comparison

Feature	fork() (Process)	pthread (Thread)
Memory	Separate memory space	Shared memory
Resource Usage	Heavy (full process copy)	Lightweight
Communication	Complex (IPC needed)	Simple (shared variables)
Safety	High (isolated)	Requires synchronization
Speed	Slower creation	Faster creation
Best For	Independent tasks	Shared data tasks

3. Method 1: Using fork() - Process-Based Concurrency

What is fork()?

fork() is a system call that creates a new process by duplicating the calling process.

pid_t fork(void);

Returns:

- **0** → In child process
- **Positive PID** → In parent process (child's process ID)
- **-1** → Error

How fork() Works

```

int pid;
int status = 0;

pid = fork();
if (pid > 0) {
    /* parent */
    ....
    pid = wait(&status);
} else {
    /* child */
    ....
    exit(status);
}

```

The **fork** syscall returns a zero to the child and the child process ID to the parent

Parent uses **wait** to sleep until child returns and

Fork creates an exact copy of the parent process

Wait variants allow wait on a specific child or

Child process passes status back to parent on **exit**, to report success/failure

Key Points:

- Child gets copy of parent's memory
- Both processes run independently
- Changes in one don't affect the other
- Child inherits open file descriptors (like sockets)

Concurrent Server Logic with fork()

Main Server Process (Parent):

1. Create a UDP socket.
2. Bind to a port number.
3. Continuously receive messages from clients.
4. On receiving a message:
 - a. Call fork() to create a child process.
 - b. The **child** handles that message and replies.
 - c. The **parent** continues listening for new clients.

4. Method 1 Server Code Explanation

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define PORT 9000
#define BUFFER_SIZE 1024

int main() {
    int sockfd;
    struct sockaddr_in serverAddr, clientAddr;
    char buffer[BUFFER_SIZE];
    socklen_t addr_size = sizeof(clientAddr);
    pid_t childpid;

    // Create UDP socket
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sockfd < 0) {
        perror("Socket creation failed");
        exit(1);
    }
    printf("Server socket created successfully.\n");

    // Configure server address
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(PORT);
```

```

serverAddr.sin_addr.s_addr = INADDR_ANY;

// Bind socket to IP/Port
if (bind(sockfd, (struct sockaddr *)&serverAddr, sizeof(serverAddr)) < 0) {
    perror("Bind failed");
    close(sockfd);
    exit(1);
}
printf("Server bound to port %d successfully.\n", PORT);
printf("Concurrent UDP Server (fork version) running...\n\n");

while (1) {
    int n = recvfrom(sockfd, buffer, BUFFER_SIZE - 1, 0,
                    (struct sockaddr *)&clientAddr, &addr_size);
    buffer[n] = '\0';

    printf("Message received from %s:%d → %s\n",
          inet_ntoa(clientAddr.sin_addr),
          ntohs(clientAddr.sin_port),
          buffer);

    // Create a new process for each client
    if ((childpid = fork()) == 0) {
        char reply[BUFFER_SIZE];
        sprintf(reply, "Server (PID %d) received: %s", getpid(), buffer);

        sendto(sockfd, reply, strlen(reply) + 1, 0,
              (struct sockaddr *)&clientAddr, addr_size);

        printf("Reply sent to %s:%d\n",
              inet_ntoa(clientAddr.sin_addr),
              ntohs(clientAddr.sin_port));

        close(sockfd);
        exit(0);
    }
}

close(sockfd);
return 0;
}

```

Explanation of fork() Server

1. **Socket Creation** – A UDP socket is created using `socket(AF_INET, SOCK_DGRAM, 0)`.
2. **Address Setup** – `serverAddr` structure is initialized with the port 9000 and IP `INADDR_ANY`.
3. **Binding** – The socket is bound to the specified IP and port using `bind()`.
4. **Listening Loop** – The server continuously waits for datagrams using `recvfrom()`.

5. **fork() Call** – When a message arrives, the server forks a **child process**.
6. **Child Process Work** – The child prepares a reply message and sends it to that specific client using `sendto()`.
7. **Parent Process Behavior** – The parent immediately returns to the listening loop, ready for another client message.
8. **Concurrent Behavior** – Each datagram is handled by its own process, allowing parallel client handling.

5. Method 2: Using Threads - Thread-Based Concurrency

What is Multithreading?

Multithreading divides a process into multiple lightweight execution units (threads).

All threads share the same memory and resources, making them more efficient for concurrent servers.

Key Features

- Each thread runs independently but shares memory with others.
- Thread creation is faster than process creation.
- Communication between threads is simple (shared variables).
- Requires synchronization if multiple threads access shared data.

Concurrent Server Logic with Threads

Main server thread:

1. Create UDP socket and bind to port.
2. Continuously receive client datagrams.
3. For each message, create a **new thread**.
4. Each thread handles one client request and sends a reply.

6. Method 2 Server Code Explanation

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include <arpa/inet.h>

#define PORT 9001
#define BUFFER_SIZE 1024

struct client_info {
    int sockfd;
    struct sockaddr_in clientAddr;
    char buffer[BUFFER_SIZE];
};
```

```

void *handle_client(void *arg) {
    struct client_info *info = (struct client_info *)arg;

    printf("Handling client %s:%d\n",
        inet_ntoa(info->clientAddr.sin_addr),
        ntohs(info->clientAddr.sin_port));

    char reply[BUFFER_SIZE];
    sprintf(reply, "Server (Thread) received: %s", info->buffer);

    sendto(info->sockfd, reply, strlen(reply) + 1, 0,
        (struct sockaddr *)&info->clientAddr, sizeof(info->clientAddr));

    free(info);
    pthread_exit(NULL);
}

int main() {
    int sockfd;
    struct sockaddr_in serverAddr, clientAddr;
    socklen_t addr_size = sizeof(clientAddr);
    pthread_t tid;

    // Create UDP socket
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sockfd < 0) {
        perror("Socket creation failed");
        exit(1);
    }
    printf("Server socket created successfully.\n");

    // Configure server address
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(PORT);
    serverAddr.sin_addr.s_addr = INADDR_ANY;

    // Bind socket
    if (bind(sockfd, (struct sockaddr *)&serverAddr, sizeof(serverAddr)) < 0) {
        perror("Bind failed");
        close(sockfd);
        exit(1);
    }
    printf("Server bound to port %d successfully.\n", PORT);
    printf("Concurrent UDP Server (thread version) running...\n\n");

    while (1) {
        struct client_info *info = malloc(sizeof(struct client_info));
        info->sockfd = sockfd;
    }

```

```

    int n = recvfrom(sockfd, info->buffer, BUFFER_SIZE - 1, 0,
                    (struct sockaddr *)&info->clientAddr, &addr_size);
    info->buffer[n] = '\0';

    pthread_create(&tid, NULL, handle_client, info);
    pthread_detach(tid);
}

close(sockfd);
return 0;
}

```

Explanation of pthread Server

1. **Socket Creation** – Creates a UDP socket using `socket(AF_INET, SOCK_DGRAM, 0)`.
2. **Address Binding** – Binds the socket to port 9001 and IP `INADDR_ANY`.
3. **Waiting for Clients** – The server continuously receives datagrams with `recvfrom()`.
4. **Dynamic Allocation** – Allocates memory for each client's information (`struct client_info`).
5. **Thread Creation** – Creates a thread using `pthread_create()` to handle that specific client's message.
6. **Thread Handler** – The thread sends a reply back to the same client and terminates.
7. **Automatic Cleanup** – `pthread_detach()` ensures thread resources are released when done.
8. **Concurrent Behavior** – Multiple threads can process multiple clients at the same time.

7. Advanced Method: Using Thread Pool for Efficient UDP Concurrency

The Problem with Creating a Thread per Message

In the previous section, the server created a **new thread for every incoming UDP message**. While this works correctly for demonstrating concurrency, it becomes inefficient when the number of messages increases rapidly.

UDP is **connectionless**, meaning:

- Each message (datagram) is independent.
- There is no persistent “client connection” like TCP.
- Messages can arrive in bursts from different clients simultaneously.

If we create a new thread for **every incoming datagram**, the system may:

- Run out of memory due to excessive thread creation.
- Spend more CPU time managing threads than processing messages.

To solve this, we use a more efficient technique called a **Thread Pool**.

What is a Thread Pool?

A **thread pool** is a collection of pre-created worker threads that stay alive to process incoming messages.

Instead of creating a new thread for every datagram:

1. The **main thread** receives messages and puts them into a **shared task queue**.
2. Worker threads (in the pool) continuously fetch messages from this queue.
3. Each worker processes the message and sends a reply.

This design is **faster**, **more scalable**, and **prevents resource exhaustion**.

Here's how the basic flow of thread pooling goes:

Main Thread:

```
while True:
    data, client_addr = recvfrom()
    add (data, client_addr) to task_queue
```

Worker Threads (N threads):

```
while True:
    data, client_addr = task_queue.get()
    process data
    send response back to client_addr
    task_queue.task_done()
```

1. The server starts with **N** worker threads
2. The main thread continuously listens for packets
3. When a new packet arrives:
 - i. The main thread places the packet and the client address in a queue
4. Each worker thread runs in a loop:
 - i. It takes a packet from the queue
 - ii. Processes the message and sends a response
 - iii. Then waits for the next task

Server Code (Using Thread Pool):

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include <arpa/inet.h>
```

```

#define PORT 9002
#define BUFFER_SIZE 1024
#define THREAD_POOL_SIZE 4
#define QUEUE_SIZE 10

// Structure representing a task (UDP message)
struct task {
    char buffer[BUFFER_SIZE];
    struct sockaddr_in clientAddr;
    socklen_t addr_size;
};

// Shared task queue
struct task queue[QUEUE_SIZE];
int front = 0, rear = 0, count = 0;

// Synchronization variables
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

int sockfd;

// Worker thread function
void *worker_thread(void *arg) {
    while (1) {
        struct task current_task;

        // Wait for available task
        pthread_mutex_lock(&mutex);
        while (count == 0)
            pthread_cond_wait(&cond, &mutex);

        // Dequeue the task
        current_task = queue[front];
        front = (front + 1) % QUEUE_SIZE;
        count--;
        pthread_mutex_unlock(&mutex);

        // Process the received message
        printf("Worker thread handling client %s:%d → %s\n",
            inet_ntoa(current_task.clientAddr.sin_addr),
            ntohs(current_task.clientAddr.sin_port),
            current_task.buffer);
    }
}

```

```

        // Prepare and send reply
        char reply[BUFFER_SIZE];
        sprintf(reply, "Processed by thread: %s", current_task.buffer);
        sendto(sockfd, reply, strlen(reply) + 1, 0,
                (struct sockaddr *)&current_task.clientAddr,
                current_task.addr_size);
    }
    return NULL;
}

int main() {
    struct sockaddr_in serverAddr, clientAddr;
    socklen_t addr_size = sizeof(clientAddr);
    char buffer[BUFFER_SIZE];
    pthread_t pool[THREAD_POOL_SIZE];

    // 1. Create UDP socket
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sockfd < 0) {
        perror("Socket creation failed");
        exit(1);
    }

    // 2. Configure server address
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(PORT);
    serverAddr.sin_addr.s_addr = INADDR_ANY;

    // 3. Bind socket
    if (bind(sockfd, (struct sockaddr *)&serverAddr, sizeof(serverAddr)) < 0) {
        perror("Bind failed");
        close(sockfd);
        exit(1);
    }

    printf("UDP Server with Thread Pool running on port %d...\n", PORT);

    // 4. Create worker threads
    for (int i = 0; i < THREAD_POOL_SIZE; i++) {
        pthread_create(&pool[i], NULL, worker_thread, NULL);
    }

    // 5. Main loop to receive and enqueue messages
    while (1) {
        int n = recvfrom(sockfd, buffer, BUFFER_SIZE - 1, 0,

```

```

        (struct sockaddr *)&clientAddr, &addr_size);
    buffer[n] = '\0';

    pthread_mutex_lock(&mutex);

    // If queue has space, enqueue task
    if (count < QUEUE_SIZE) {
        strcpy(queue[rear].buffer, buffer);
        queue[rear].clientAddr = clientAddr;
        queue[rear].addr_size = addr_size;
        rear = (rear + 1) % QUEUE_SIZE;
        count++;
        pthread_cond_signal(&cond);
    } else {
        printf("Queue full, message dropped from %s:%d\n",
            inet_ntoa(clientAddr.sin_addr),
            ntohs(clientAddr.sin_port));
    }

    pthread_mutex_unlock(&mutex);
}

close(sockfd);
return 0;
}

```

Step-by-Step Explanation of Code

1. Socket Creation

- A UDP socket is created using `socket(AF_INET, SOCK_DGRAM, 0)`.

2. Server Address Setup

- The server is bound to port 9002 and listens on all network interfaces using `INADDR_ANY`.

3. Worker Thread Creation

- A fixed number (`THREAD_POOL_SIZE`) of worker threads are created at startup.
- Each thread runs the `worker_thread()` function and waits for new tasks.

4. Main Thread Operation

- The main thread continuously receives datagrams from clients using `recvfrom()`.
- Each received message, along with the client's address, is stored as a **task** in a **shared queue**.

5. Task Queue and Synchronization

- The queue is protected by a mutex (`pthread_mutex_t`) to ensure thread-safe operations.
- When a message is added, the main thread signals worker threads using a condition variable (`pthread_cond_signal()`).

6. Worker Thread Execution

- A worker thread wakes up, removes a message from the queue, and processes it.

- It prepares a response and sends it back to the correct client using `sendto()` .

7. Continuous Processing

- After completing a task, the thread waits for the next message automatically.
- The server keeps running indefinitely, handling multiple clients concurrently without creating new threads per message.

Why Thread Pooling is Better:

- **Thread Creation:**

Without thread pooling, a new thread is created for every incoming message, which consumes a lot of resources.

With thread pooling, a fixed number of threads are created once and reused for all messages.

- **Performance:**

Without thread pooling, performance becomes slow under heavy traffic because of continuous thread creation and destruction.

With thread pooling, performance remains fast and consistent even during high message loads.

- **Resource Usage:**

Without thread pooling, the number of threads can grow uncontrollably, potentially crashing the system.

With thread pooling, the number of threads is fixed and controlled, preventing overload.

- **Scalability:**

Without thread pooling, scalability is poor because system resources are quickly exhausted.

With thread pooling, scalability is excellent since threads are reused efficiently.

- **Complexity:**

Without thread pooling, the design is simple but inefficient for real-world scenarios.

With thread pooling, the design is slightly more complex but much more efficient and robust.

8. Multithreading vs Thread pooling

Feature	Threading per message	Thread Pool
Thread Creation	New thread for every message	Fixed number of threads
Performance	Poor under heavy load	Scales well
Resource Usage	High (many threads)	Controlled and predictable
Implementation	Simple	Slightly more complex (needs queue)
Suitable For	Light workloads	Queue overflow (manageable)

In essence, creating a new thread per message is like hiring a new employee every time you get a client, while Thread pooling is like having an effective roster of workers you can rely on to serve any client needs

Note:

Although this lab demonstrates concurrency in UDP using threads and thread pooling, a similar approach can also be implemented using **processes with fork()** (known as a preforked process pool). However, the thread-based method is generally preferred for UDP due to lower overhead and better scalability.

9. Summary

- Iterative servers handle one client at a time → cause delays.
- Concurrent servers handle multiple clients simultaneously.
- Process-based concurrency (fork)
 - Creates a new process per client.
 - Each process has separate memory.
 - Safer but heavier on system resources.
- Thread-based concurrency (pthread)
 - Creates a new thread per client.
 - Threads share memory → faster communication.
 - Lightweight but needs synchronization.
- Thread pooling
 - Pre-creates a fixed number of threads.
 - Tasks placed in a shared queue.
 - Improves performance and scalability.
- Comparison
 - fork() → isolated, slower.
 - pthread → faster, shared memory.
 - Thread pool → most efficient for many clients.

Good luck with the assignment

